# Sicherheit von PDF-Dokumenten

## Grundlagenpraktikum zur IT-Sicherheit (WiSe 2023/2024)

Lehrstuhl für Netz- und Datensicherheit
Ruhr-Universität Bochum

Nico S**** Sabri Öztürk

1. Mai 2026

**Zusammenfassung**

In diesem Versuch wird gezeigt, wie PDF-Dokumente strukturiert sind. Anhand einiger Beispiele, werden verschiedene unsichere Aspekte von PDF-Dokumenten und wie diese ausgenutzt werden können dar gestellt. Weiterhin wird ein Angriff durchgeführt, der eine veränderte, signierte Datei weiter als gültig signiert gelten lässt.

# Inhaltsverzeichnis

**1 Einleitung** **1**

1.1 Glossar . . . . . 1

**2 Aufbau** **2**

**3 Durchführung** **3**

3.1 Setup . . . . . 3

3.2 Fix Me! . . . . . 4

3.3 Insecure Features . . . . . 6

3.4 Incremental Savings Attack . . . . . 8

**4 Responsible disclosure** **11**

**5 Zusammenfassung und Fazit** **12**

**6 LLM** **13**

# 1 Einleitung

In der heutigen Zeit werden Dokumente meist digital übertragen. Darunter kann die Sicherheit jedoch leiden. Wo früher eine Siegel oder eine Unterschrift ein Dokument für gültig befunden hat, kann heute jeder ein Unterschriebenes PDF-Dokument am Computer kopieren. So kann unter jedes beliebige Dokument die Unterschrift eines Politikers oder eines jeden anderen Menschen gesetzt werden. Um Dateien weiterhin vertrauen zu können und den Absender authentifizieren zu können, werden digital Signaturen eingesetzt.

Diese Signaturen können durch unterschiedliche Verfahren eine Person authentifizieren. Eine solche Signatur sollte nicht gefälscht werden können. Doch durch falsche Implementierungen von Signaturen oder Fehlern in Programmen die Signaturen verwalten, kann es dennoch möglich sein, eine Identität zu stehlen.

Um zu verstehen wie solche Fehler in PDF-Dokumenten auftreten können, muss erst einmal die Struktur einer PDF-Dateie verstanden werden. Der Grundstruktur eines PDF-Dokuments zu verändern kann durch verschiedene Programme erreicht werden.

In den folgenden Versuchen wird die Struktur von PDF-Dokumenten analysiert um Fehler zu erkennen und ein Dokument wieder lesbar zu machen. Darauf folgen Funktionen von PDF-Dokumenten, welche ein Angreifer ausnutzen kann, um Informationen zu stehlen und zuletzt wie ein Dokument, dass eine gültige Signatur hat, verändert werden kann, ohne dass die Signatur ungültig wird.

## 1.1 Glossar

**PDF** Portable Document Format

**Integrität** Daten werden nicht unautorisiert geändert

**Authentizität** Daten oder Subjekte werden als echt und glaubwürdig befunden

# 2 Aufbau

Die folgenden Versuche wurden im Netzlabor ID 2/168 unternommen.
Wir führen den Versuch zu zweit als Gruppe 43 am PC mit der IP-Adresse 192.168.1.42 durch.
Das Betriebsystem ist ein Ubuntu 22.04 LTS.

Es wurden verschiedene Programme und Befehle genutz. Diese sind:

| **Software** | **Version** |
|---|---|
| PDF Studio | 12.0.6 |
| Notepad++ | 8.6 |
| snap | 2.60.4 |

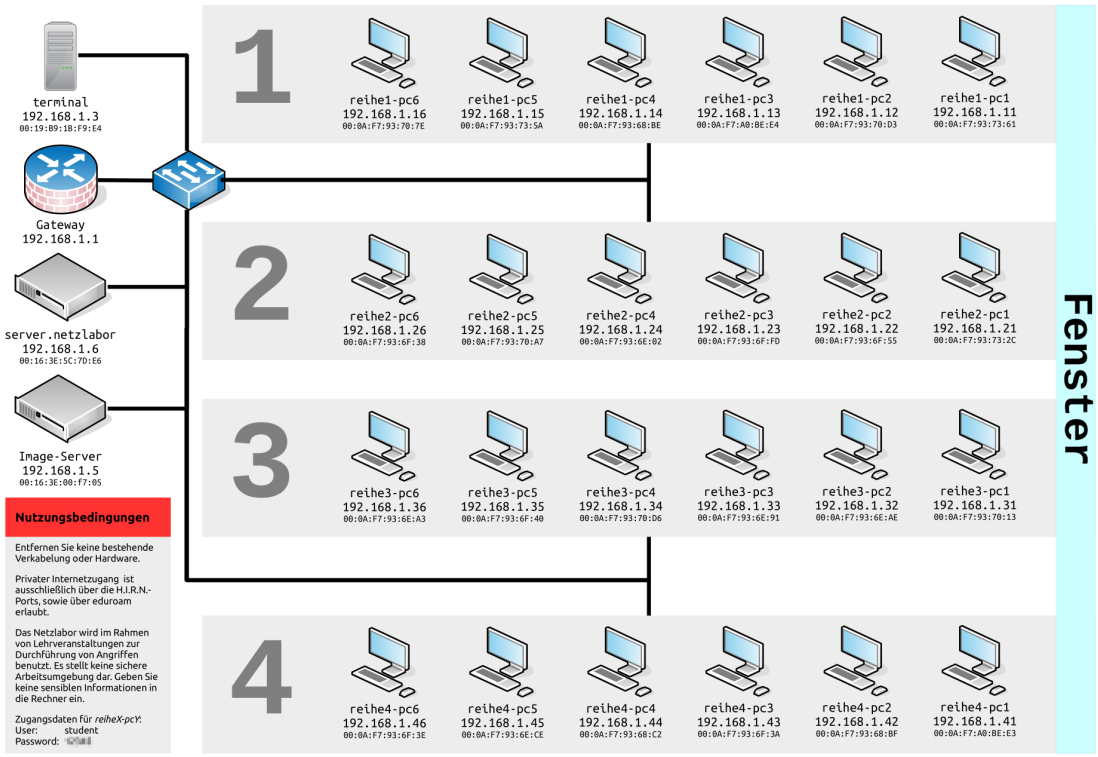


Abbildung 1: Netzplan Raum ID 2/168, Stand 01.05.2017

Die Aufgabenstellung [3] wurde bereutgestellt übe die Veranstaltungsseite [2]

# 3 Durchführung

Im Folgenden werden die einzelnen Aufgaben durchgeführt, erläutert und das Ergebnis präsentiert.

## 3.1 Setup

Um PDF-Dokumente einsehen zu können, muss auf jedem Gerät Software dafür installiert werden. Heutzutage können viele Browser PDF-Dokumente anzeigen und sogar in einigen Fällen Kommentare und Anmerkungen in dem Dokument einfügen. Doch wenn der volle Umfang an Funktionalität von PDF-Dokumenten genutzt werden soll, ist dafür besondere Software nötig. Eine solche Software ist *PDF Studio*. Mit dieser können Dokumente überprüft und kommentiert, Formulare ausgefüllt, ganze Dateien mit einem Passwort versehen und ein neues Dokument aus anderen zusammengesetzt werden [1].

In diesem Versuch wird *PDF Studio* vorallem dafür genutzt, digitale Zertifikate auf ihre Gültigkeit zu überprüfen.

Um *PDF Studio* zu installieren, ist eine Datei gegeben, welche die Installation durchführt. Diese ist in dem Verzeichnis */home/student/Praktikum/PDF/* unter dem Namen *PDFStudio_v12_0_6_linux64.sh* zu finden. Die Installation erfolgt über das Terminal durch das ausführen der Datei.

```
student@Reihe4-PC2:~$ cd /home/student/Praktikum/PDF/
student@Reihe4-PC2:~/Praktikum/PDF/$ ./PDFStudio_v12_0_6_linux64.sh
```

Sobald die Datei ausgeführt ist, wird eine graphische Oberfläche für Einstellungen der Installation zur verfügung gestellt. Die Installation wird abgeschlossen durch: Deutsch -Akzeptieren -Zielordner unverändert lassen -Desktopsymbol erstellen lassen. Zuletzt wird die stadard Version von *PDF Studio* getestet um kostenlosen Zugang zu dem Programm zu bekommen.

Da *PDF Studio* nun installiert ist, kann ein Zertifikat importiert werden. Dieses Zertifikat gehört zu dem Namen *Bob* und liegt in dem Verzeichnis */home/student/Praktikum/PDF/Incremental Savings Attack/* mit dem Namen *Bob Smith.cer*.

Die letzte Vorbereitung für diesen Versuch besteht darin, dass *Notepad++* installiert werden muss. Mit diesem Text-Editor lassen sich die Strukturen von PDF-Dokumenten, also *Header*, *Body*, *XRef* und *Trailer* anzeigen und verändern. Da *Notepad++* eigentlich ein Programm für Windows ist, kann es unter Linux nicht direkt installiert werden. Über *Snap* kann jedoch mit der Hilfe von *WINE* ein Windows-Betriebsystem emuliert werden, sodass auch Windows-Programme unter Linux ausgeführt werden können. Der Befehl um *Notepad++* zu installieren lautet: *sudo snap install notepad-plus-plus*. Nun kann *Notepad++* über die Konsole aufgerufen werden.

## 3.2 Fix Me!

Jeder Dateityp der von einem Computer gelesen werden soll, hat eine bestimmte Struktur, die beibehalten werden muss, damit der Computer die Datei auch richtig Lesen und Anzeigen kann. So haben auch PDF-Dokumente eine ihnen zugrunde liegende Struktur. Um mit dieser Struktur vertraut zu werden, kann ein PDF-Dokument in *Notepad++* geöffnet werden. Dort wird eine für Menschen lesbare Version der Binärdaten des Dokuments angezeigt.

Ein PDF-Dokument, welches sich nicht mehr öffnen lässt liegt in dem Verzeichnis */home/student/Praktikum/PDF/Fix Me!/* unter dem Namen *FixMe.pdf*. Die Datei enthält fünf Fehler, die gefunden und korrigiert werden müssen, damit die Datei wieder lesbar ist. Zu beginn kann die Datei in *Notepad++* geöffnet werden.

Ein Bild der orginal Datei in *Notepad++* findet sich unter Abbildung 3. Die verbesserte Datei ist in Abbildung 4 dargestellt.

Es kann festgestellt werden, dass bereits am Anfang der Datei, in Zeile 4, ein Fehler ist. Das erste Objekt muss ein *Catalog* sein. Dieses ist das Wurzel Objekt und des Dokuments und legt den weiteren Aufbau fest.

Der nächste Fehler findet sich in Zeile 10 in Objekt drei. Hier fehlt eine Angabe in */Contents*, die auf das nächste Objekt verweist. Der Fehler kann durch das Einfügen der Zahlen '4' und '0' für das nächste Objekt behoben werden.

Danach wird in Zeile 14 ein *stream* gestartet. Dieser wurde jedoch nicht beendet. Der *stream* sollte nach *ET* in Zeile 19 beendet werden. Dafür wird *endstream* genutzt, um auch diesen Fehler zu beheben.

Die nächsten Fehler liegen am Ende des Dokuments. Im *trailer* in Zeile 41 wird nicht die genaue Position des Wurzelobjekts angegeben. Dies lässt sich durch das erweitern von */Root 0 R* zu */Root 1 0 R* bewerkstelligen.

Der letzte Fehler liegt darin, dass die letzt Zeile des Dokuments nicht *%%EOF* ist. 'EOF' steht für *End Of File* und muss in jedem PDF-Dokuemtn vorhanden sein. Wenn auch dieser Fehler behoben ist, kann das Dokument gespeichert und nun erfolgreich geöffnet werden.

Die Datei enthält eine einzige Zeile in der *Hello World!* steht.

**Hello World!**

Abbildung 2: Inhalt der Datei 'FixMe.pdf'

FixMe.pdf

```
%PDF-1.4
%¿÷¢þ
1 0 obj
<< /Pages 2 0 R /Type / /Version /1.4 >>
endobj
2 0 obj
<< /Count 1 /Kids [ 3 0 R ] /Type /Pages >>
endobj
3 0 obj
<< /Contents  R /MediaBox [ 0.0 0.0 612.0 792.0 ] /Parent 2 0 R /Resources 5 0 R /Type /Page >>
endobj
4 0 obj
<< /Length 45 >>
stream
BT
/F1 12 Tf
100 700 Td
(Hello World!) Tj
ET

endobj
5 0 obj
<< /Font 6 0 R >>
endobj
6 0 obj
<< /F1 7 0 R >>
endobj
7 0 obj
<< /BaseFont /Helvetica-Bold /Encoding /WinAnsiEncoding /Subtype /Type1 /Type /Font >>
endobj
xref
0 8
0000000000 65535 f
0000000015 00000 n
0000000078 00000 n
0000000137 00000 n
0000000251 00000 n
0000000345 00000 n
0000000378 00000 n
0000000409 00000 n
trailer << /Root  0 R /Size 8 /ID [<1fdc264c24377f037dc7c2587f9c9aa8><6ad9823a6bedd8bb26ce435a9a55c37e>] >>
startxref
511
%%
```

Abbildung 3: Original der Datei 'FixMe.pdf'

FixMe.pdf

```
%PDF-1.4
%¿÷¢þ
1 0 obj
<< /Pages 2 0 R /Type /Catalog /Version /1.4 >>
endobj
2 0 obj
<< /Count 1 /Kids [ 3 0 R ] /Type /Pages >>
endobj
3 0 obj
<< /Contents 4 0 R /MediaBox [ 0.0 0.0 612.0 792.0 ] /Parent 2 0 R /Resources 5 0 R /Type /Page >>
endobj
4 0 obj
<< /Length 45 >>
stream
BT
/F1 12 Tf
100 700 Td
(Hello World!) Tj
ET
endstream
endobj
5 0 obj
<< /Font 6 0 R >>
endobj
6 0 obj
<< /F1 7 0 R >>
endobj
7 0 obj
<< /BaseFont /Helvetica-Bold /Encoding /WinAnsiEncoding /Subtype /Type1 /Type /Font >>
endobj
xref
0 8
0000000000 65535 f
0000000015 00000 n
0000000078 00000 n
0000000137 00000 n
0000000251 00000 n
0000000345 00000 n
0000000378 00000 n
0000000409 00000 n
trailer << /Root 1 0 R /Size 8 /ID [<1fdc264c24377f037dc7c2587f9c9aa8><6ad9823a6bedd8bb26ce435a9a55c37e>] >>
startxref
511
%%EOF
```

Abbildung 4: Verbesserung der Datei 'FixMe.pdf'

## 3.3 Insecure Features

Wir öffnen die Datei 'SymptomeFormular.pdf', in der ein eingebetteter Angriff vorhanden ist, sowohl mit PDF Studio Pro als auch mit Notepad++. Auf Zeile 56 entdecken wir eine URL, an die der Inhalt des PDFs gesendet wird. Dieser Angriff kann als Phishing eingestuft werden. Wenn die Daten eingegeben werden, wird das PDF automatisch an die genannte URL, `https://evil.com`, gesendet, wodurch die persönlichen Informationen eines Patienten an diese Seite übermittelt werden. Dadurch könnte die Seite später bösartige Absichten damit verfolgen. Der Angreifer nutzt dabei Objekte 5 und 6 aus. Objekt 5 führt eine 'SubmitForm'-Operation aus, die die Daten des Formulars an eine externe URL sendet. Objekt 6 speichert diese Angaben, und wenn sie eingegeben werden, erfolgt automatisch die Übertragung an die bösartige URL.

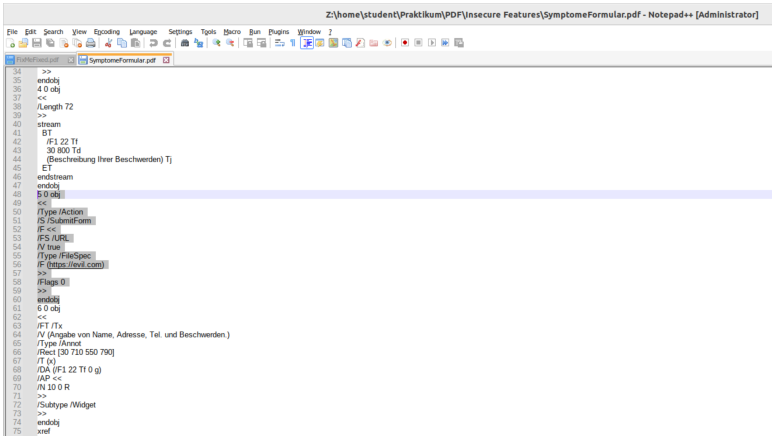


Abbildung 5: Angriff in SympomeFormular.pdf

Um die Situation zu kontrollieren, werden wir die URL mit unserer eigenen Webserver-Adresse ersetzen, um die Informationen zu empfangen, die der Angreifer normalerweise erhalten würde. Dazu erstellen wir mit Webhook unseren eigenen Webserver. Anschließend ändern wir die Zeile 56 im Dokument, indem wir die URL des Angreifers durch unsere eigene URL ersetzen.

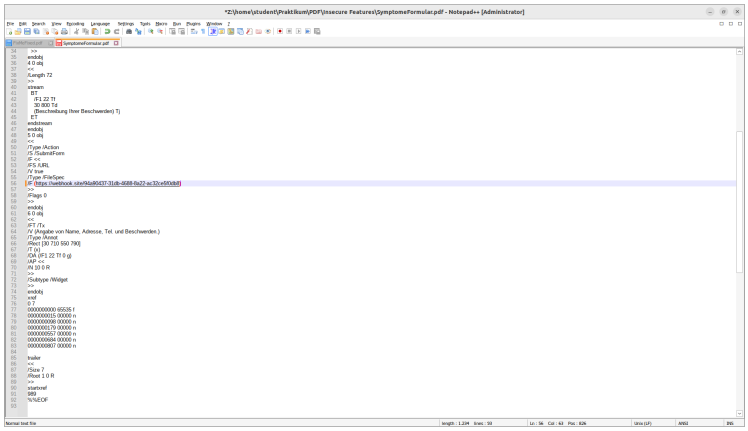

Abbildung 6: Angriff Modifiziert

Darauffolgt testen wir das ganze, in dem wir das Formular ausfüllen. Wir sehen, dass die Veränderung geklappt haben und somit die Eingaben in unserem Webserver landen.

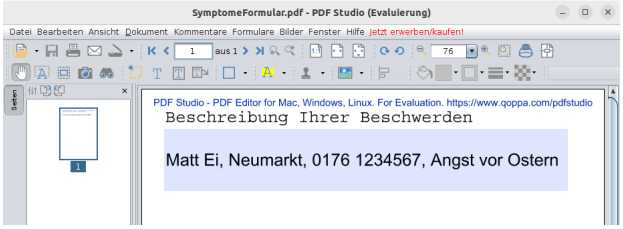


Abbildung 7: Ausgefüllte Formular

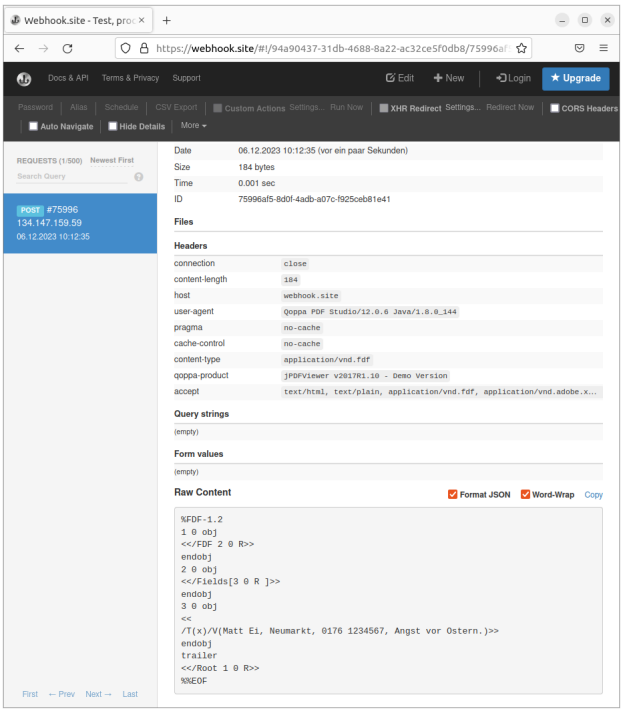


Abbildung 8: Die Eingaben erreichen unsere Webserver

Zunächst analysieren wir den Angriff. Dieser wird durch die Eingabe von Informationen ausgelöst, wobei das Opfer nicht bemerkt, dass seine Daten an einen Server gesendet werden. Mit Hilfe von Wireshark kann der Phishing-Angriff erkannt werden, indem nach ausgehenden Netzwerkpaketen gesucht wird. Die wichtigsten Informationen, die auf dem Webserver empfangen werden, sind die vom Benutzer eingegebenen Daten. Die übrigen Informationen sind für den Angriff nicht relevant. Die für die Übermittlung verwendete HTTP-Methode ist die 'POST'-Methode. Das Übertragungsformat liegt im FDF-Format bzw. Forms Data Format.

## 3.4 Incremental Savings Attack

Im nächsten Schritt wollen wir auf PDF-Dokument 'UeberweisungSigned.pdf' einen Angriff fortzsetzen, in dem die PDF-Signatur gültig bleibt, obwohl der Inhalt verändert wird. In dem Datei liegt eine Überweisung von Bob an Alice. Wir wollen den Betrag und den Empfänger des Geldes verändern. Als erstes analysieren wir das Dokument, in dem wir es mit PDF-Studio öffnen und die Signatur uns näher anschauen. Die Signatur besagt, dass das Dokument unverändert sei und somit die Signatur gültig ist.

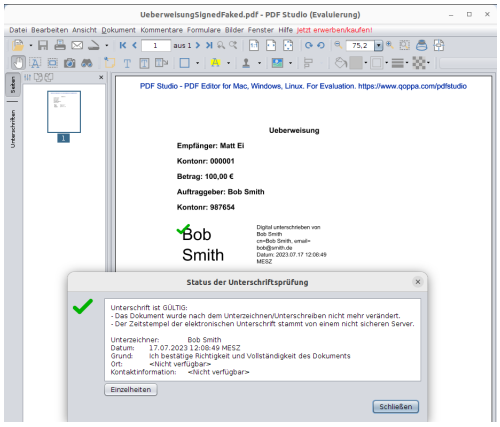


Abbildung 9: Gültige Signatur des Dokuments

Dann öffnen wir das Dokument 'UeberweisungSigned.pdf' mit Notepad++ und betrachten insbesondere die Struktur des Dokuments, insbesondere seine Signatur. Die Signatur befindet sich im Objekt 23. Sie wurde am 17. Juli 2023 von Bob ausgestellt und handelt sich um eine PKCS7-Signatur. Des Weiteren befinden sich Textelemente in den Objekten 4 bis einschließlich 12. Auf den ersten Blick ist es etwas verwirrend, dass es zwei '%%EOF'-Markierungen gibt. Die eine beendet alle Objekte der PDF selbst, aber dann kommen optional Body-Updates ins Spiel, die ebenfalls am Ende '%%EOF' enthalten, was auch dieses Mal der Fall ist.

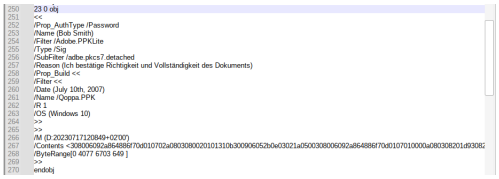


Abbildung 10: Signatur von UeberweisungSigned.pdf im Notepad++ betrachtet

Zunächst manipulieren wir den Betrag, der versendet werden soll, um das Verhalten der Signatur zu beobachten. Die Signatur besagt, dass die Unterschrift ungültig ist und dass das Dokument nach der Erstellung der Signatur verändert wurde. Dadurch wird der Anspruch auf Integrität und Authentizität aufgehoben. Dies verdeutlicht die Funktionsweise einer Signatur. Sie bewahrt die Integrität und Authentizität, in dem sie, bei einem Ausfall dieser Aspekte entweder Warnungen generiert oder den Nutzern den veränderten Inhalt signalisiert.

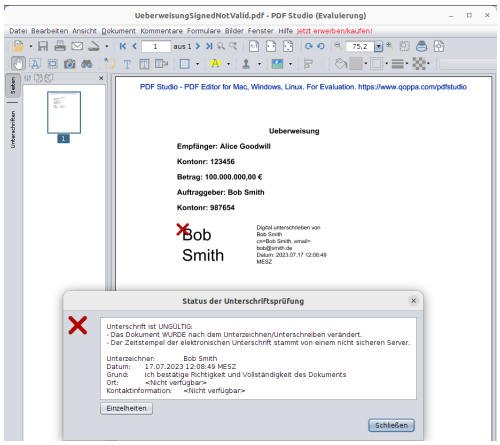


Abbildung 11: Der Einfluss von dem Veränderung des Betrags auf die Signatur

Nun bereiten wir uns auf den Angriff vor. Um den Angriff erfolgreich durchzuführen müssen wir einige Informationen in Erfahrung bringen. Das von uns genutzte PDF Studio Pro hat die Version 12.0.6 und ist laut pdf-insecruity.org verwundbar, da die naheliegendste verwundbare Version 12.0.7 ist. Da eine spätere Version verwundbar ist, können wir davon ausgehen, dass auch unsere Version die Schwachstelle enthält. Die ISA Proof-of-Concept Datei enthält drei End-of-File's (%%EOF) welche für verschiedene Teile der Datei stehen. Das erste EOF beendet die originale Datei, das zweite beendet die angefügte Signatur. Für uns ist jedoch das dritte interessant, da in diesem Abschnitt die Inhalte des originalen Dokuments überschrieben werden und durch eigene Informationen ausgetauscht werden. Trotz der Änderungen bleibt die Signatur gültig. Wenn wir das Dateiende betrachten, sehen wir, dass es Body-Updates, neue Signatur Objects, eine neue Xref-Tabelle und ein Trailer enthalten sind. Für unseren Angriff werden wir die Variante 3 einsetzen, wo Body-Updates und Trailer hinzugefügt werden.

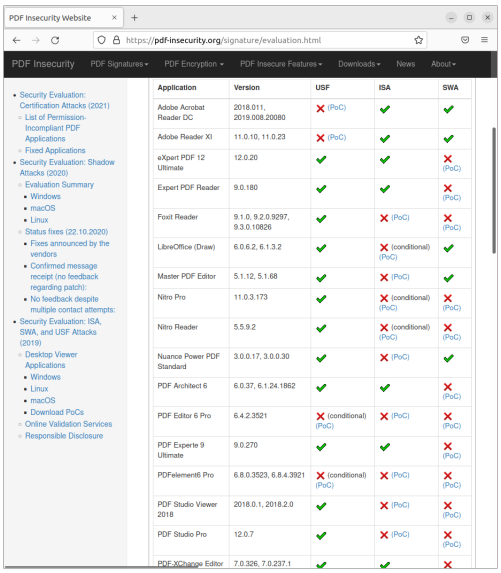


Abbildung 12: PDF Studio Pro ist gegenüber ISA verwundbar

Im letzten Schritt wollen wir eine Incremental Saving Attack auf die bisher unveränderten 'UeberweisungSigned.pdf' durchführen, wobei die Signatur weiterhin erfolgreich validiert wird, obwohl der Inhalt ein anderer ist. Der Inhalt soll so verändert werden, dass der Empfänger-Name und die Empfänger-Kontonummer verändert werden. Dazu kopieren wir die kompletten Objektdefinitionen mit der Reinfolge für Name, Kontonummer und den gesamten Trailer, am Ende der Datei. Dann ersetzen wir den Text der Objekt 8 mit 'Empfänger: Matt Ei' und den Text der Objekt 9 mit 'Kontonr: 000001'. Danach löschen wir 'prev', das nach der 'Size26' kommt, damit die alte XRef-Tabelle nicht gefunden werden kann. Ganz am Ende vor dem '%%EOF' fügen wir 'startxref' hinzu, damit wird die Anzeige des Dokuments als gültiges PDF-Dokument signalisiert, ohne jedoch eine aktualisierte Tabelle anzufügen. Das Update muss an das Ende der Datei eingefügt werden, denn in einem anderen Stelle, die Signatur nicht mehr validiert werden würde, da das Update nicht als Body-Update anerkannt werden würde. Der 3. Angriffsart kommt zum Einsatz, denn die erste Variante aufgrund seiner Xref-Tabelle, als keine Body-Update angesehen würde. Bei dem 2. und 3. Variante könnte es sein, dass Body-Updates gar nicht gelesen würden, denn der Trailer ist das erste Element, auf welches der Reader zugreift und dann wird es von unten nach oben die weitere Objekte gelesen. Da das Body-Update sich unter dem letzten Trailer befindet, könnte die Body-Updates als keine Body-Update, sondern als eine Veränderung des original Dokuments wahrgenommen werden, was die Gültigkeit der Signatur beeinträchtigen könnte. Die ID des Objects, in dem sich der Name befindet, ist 8. Wobei die ID des Objects, in dem sich die Kontonummer des Empfängers befindet, ist 9. Wenn wir weiteren Updates vornehmen würden, würde es dazu ankommen, welches Objekt der originial Datei wir verändern möchten. Der Regel würde sein, dass die Reinfolge erhalten bleibt. Wenn wir beispielsweise Objekt mit dem ID 7 manupilieren möchten, müsste es am Anfang der Body-Updates kommen. Nun fügen wir unsere Update in das Dokument ein und öffnen das Dokument mit PDF Studio. Wir sehen, dass die Veränderungen vorgenommen worden und dass die Signatur validiert wird, wovon resultiert werden kann, dass der Angriff erfolgreich war.

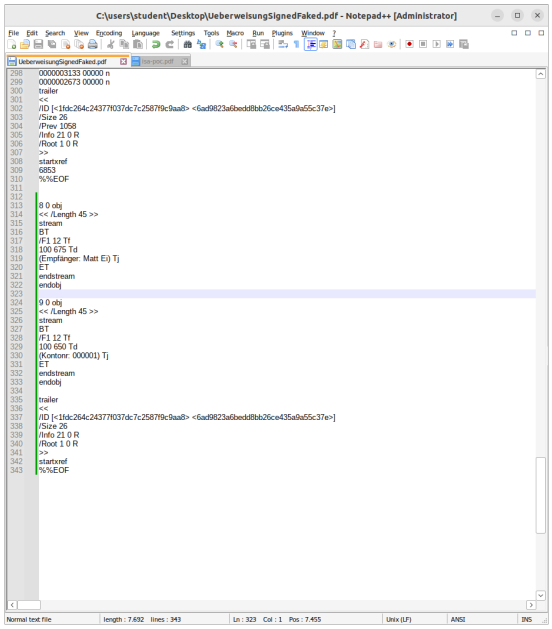


Abbildung 13: Body-Updates auf dem Dokument UeberweisungSigned.pdf

# 4 Responsible disclosure

Sehr geehrte Damen und Herren,

bei einer kürzlich erfolgten Überprüfung Ihrer Abrechnungsmethoden sind einige schwerwiegende Probleme entdeckt worden. Durch Schwachstellen in verschiedenen Programmen ist es möglich die per Signatur authentifizierten Abrechnungen in einem anderen Namen auszustellen, die Signatur jedoch nicht zu beeinträchtigen.

Dieses Problem betrifft jeden Nutzer Ihres Shops, der Abrechnungen per PDF-Dokument einreicht und kann zu erheblichen Schäden führen. Einem Angreifer ist es möglich, den Namen auf Abrechnungen auszutauschen um möglicherweise sich selbst zu bereichern. Weiterhin wird dadurch Kunden oder Mitarbeitern das ihnen zustehnde Kapital nicht ausgestellt.

Das Problem besteht darin, dass veraltete Versionen des Programms *PDF Studio* nicht gegen sogenannte *Incremental Saving Attacks* gesichert sind. Dies erlaubt einem Angreifer, Body Updates an ein PDF-Dokument anzuhängen um Inhalte zu verändern, ohne das die Signatur ungültig wird.

Als Gegenmaßnahme könnte die Zugriffberechtigung der PDF-Datei eingestellt werden, sodass nur autorisierte Benutzer write-Operation durchführen kann. Außerdem könnte eine andere Anwendung verwendet werden, die gegenüber *Incremental Saving Attacks* nicht anfällig ist.

Es wird dringend der Kontakt zu Ihren Systemadministratoren gesucht, um die notwendigen Updates und Änderungen durchzuführen. Zudem sollte dies schnellst möglich geschehen. Eine notwendige, wenn auch kurze, Frist von 7 Tagen sollte, im Hinblick auf die Dringlichkeit, eingehalten werden.

Mit freundlichen Grüßen,

Nico Steppeler und Sabri Öztürk, Berater für IT-Sicherheit

# 5 Zusammenfassung und Fazit

In diesem Versuch sind PDF-Dokumente genauer untersucht worden. Zu diesem Zwecke wurden sie in dem Editor *Notepad++* angezeigt um die grundlegende Struktur zu erforschen. Nachdem wir verstanden haben, wie PDF-Dokumente aussehen sollen, haben wir ein PDF-Dokument, welches nicht zu öffnen war, reparieren müssen. Zudem haben wir verschiedene Angriffe und Lücken in und durch PDF-Dokumente kennen gelernt

Wir haben heraus gefunden, dass persönliche Daten aus PDF-Formularen über das Internet an einen Angreifer gesendet werden können. So ist ein Identitätsdiebstahl möglich. Zudem kann der Inhalt von PDF-Dokumenten, welcher durch eine Signatur athentifiziert ist, in manchen fällen nachträglich geändert werden, ohne das die Signatur dabei ungültig wird. Dies ist durch Body Updates und veraltete Software möglich.

Aus solchen Angriffen können schwerwiegende Folgen entstehen, weshalb jeder aufpassen sollte, was er über sich selber preisgibt. Selbst beim bearbeiten von alltäglichen PDF-Dokumenten.

Da der Versuch neu war, gab es einige unstimmigkeiten. Teile des Versuches (*3.4 Incremental Savings Attack*) haben sich nur wie abtippen und 'copy-paste' angefühlt. Es musste in keinster weise selber nachgedacht werden. Wir denken, dass die Aufgabe ohne jegliche Hilfe, sehr schwer gewesen wäre, doch anstelle einer Schritt-für-Schritt-Anleitung hätten leitende Fragen gestellt werden können. Dies hätte den ansonsten sehr interessanten Versuch noch interessanter gestalten können. Das reparieren des PDF-Dokuments war lehreich. Das PDF-Dokumente auf eine solche Art und Weise dargestellt und bearbeitet werden können war neu für uns und das fehler finden eine lehr- und hilfreiche Aufgabe.

# 6 LLM

In diesem Versuch wurden keine Large-Language Models(LLM) verwendet. Die Texte sind alleine die Arbeit der Mitglieder der Gruppe 43 wie zubeginn angegeben.

# Literatur

[1] URL https://www.qoppa.com/pdfstudio/de/.

[2] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Bachelor Grundlagenpraktikum ITS: Veranstaltungsseite der Fakultät für Elektrotechnik und Informationstechnik. URL https://informatik.rub.de/studium/lehrveranstaltungen/.

[3] Ruhr-Universität Bochum Lehrstuhl für Netz-und Datensicherheit. Aufgabenstellung zum Praktikumsversuch. Bochum, 2023.